Chapter 3

PROPOSED METHODOLOGY

3.1 Overall Framework

3.1.1 System Pipeline

The proposed pipeline for aspect-based sentiment analysis of Vietnamese fintech reviews consists of seven sequential stages that transform raw Google Play reviews into actionable sentiment and aspect predictions. The stages are organized to address each of the core challenges identified in Chapter 2: informal Vietnamese text normalization, severe class imbalance, model selection across recurrent and Transformer architectures, and multi-task learning evaluation.

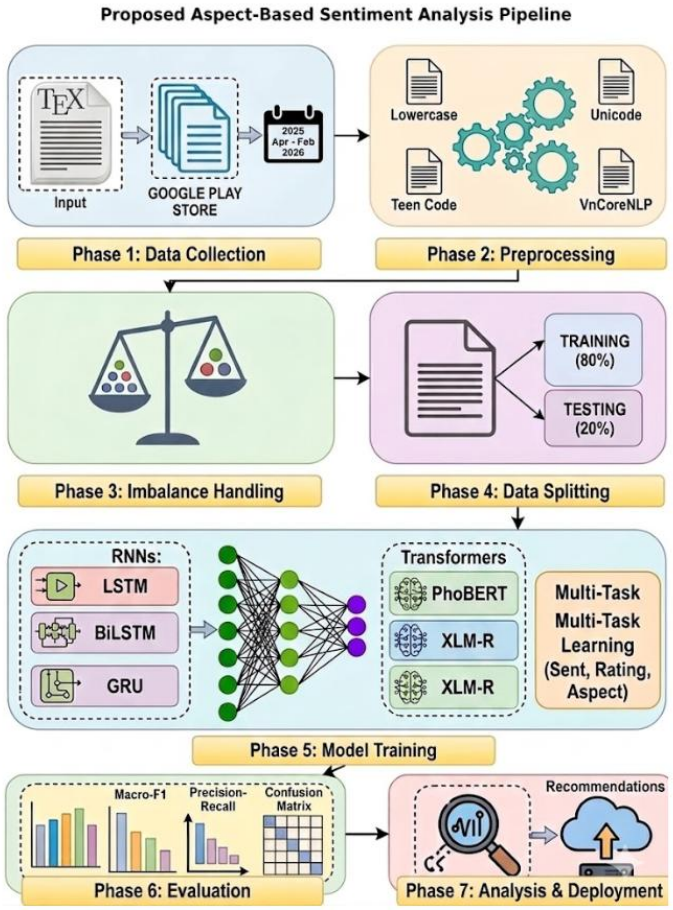

Figure 3.1: Architecture of the Proposed Aspect-Based Sentiment Analysis Pipeline

The pipeline begins with systematic data collection from the Google Play Store, followed by a multi-step preprocessing stage that normalizes Vietnamese text including teen code and emoji. The third

stage applies one of four imbalance-handling strategies before the data is split chronologically into training and test sets. Model training covers six architectures across two families ,recurrent networks and Transformer-based models ,with the option of single-task or multi-task learning configurations. The final stages evaluate all models under a consistent set of metrics and apply the best-performing model to categorize common complaint patterns across the six fintech aspect categories.

3.1.2 Workflow Description

The overall workflow is designed to ensure that each stage feeds cleanly into the next without introducing data leakage or evaluation bias. Data is split chronologically rather than randomly, preserving the temporal ordering of reviews and ensuring that the test set contains only reviews written after all training reviews. Imbalance handling is applied only to the training set. The test set is left untouched so it matches the class distribution a production system would see, and all metrics are computed on this held-out set without further tuning.

Each architecture is tested under all four imbalance strategies, producing a full grid of results that shows both the best overall performance and how each strategy interacts with model capacity. The multi-task experiments use the same train-test split and preprocessing as the single-task models, making sentiment F1 directly comparable between the two setups. Because data preparation stays constant across all experiments, any performance differences reflect actual model and strategy choices, not differences in how the data was prepared.

3.2 Data Collection

3.2.1 Review Collection Process

The dataset contains 4,325 Momo digital wallet reviews scraped from the Google Play Store over 286 days, from April 28, 2025 to February 8, 2026. Momo is Vietnam's largest digital payment platform, with over 30 million registered users as of 2024. The scraper collected each review's full text, username, star rating (1 to 5), and timestamp.

Each review has five fields: text, username, star rating, timestamp, and manually assigned labels for sentiment and aspect category. Star ratings are stored separately rather than used as sentiment labels because the two do not always match. A user can give three stars and write mostly negative text, or give five stars while complaining about a specific feature. This separation also makes it possible to use rating prediction as an independent auxiliary task in the multi-task experiments.

3.2.2 Annotation Framework

Each review received two types of labels: an overall sentiment (Positive, Negative, or Neutral) and one or more aspect tags from an eight-category fintech taxonomy. Positive reviews express satisfaction or praise, Negative reviews express dissatisfaction or complaint, and Neutral covers factual, ambiguous, or mixed reviews that do not lean clearly either way.

The eight aspect categories reflect what digital wallet users tend to talk about in their reviews:

- **Transaction**: payment processing, transfer success or failure, loading speed.
- **UI/UX**: interface design, navigation, ease of use.
- **Security**: account safety, authentication, fraud.
- **Customer Support**: interactions with support staff, complaint resolution.

- **Fee**: charges, commission rates, pricing transparency.
- **Promotion**: cashback, vouchers, reward programs.
- **Savings**: deposit and savings features.
- **General**: reviews that do not fit a specific category.

Table 3.1 shows the dataset distribution.

Table 3.1: Dataset Statistics and Label Distribution

Attribute	Count	Percentage
Total reviews	4,325	100%
Sentiment Distribution		
Negative	3,515	81.3%
Positive	578	13.4%
Neutral	232	5.4%
Star Rating Distribution		
1 star	3,271	75.6%
2–3 stars	476	11.0%
4–5 stars	578	13.4%
Text Length		
Mean	101 characters	—
Median	72 characters	—
Top Aspect Categories		
UI/UX	899	20.8%
Transaction ,UI/UX	692	16.0%
General	685	15.8%
Transaction	441	10.2%

3.3 Data Preprocessing

3.3.1 Preprocessing Pipeline

Preprocessing consists of five steps, applied in a fixed order so that all models and all data folds receive the same input.

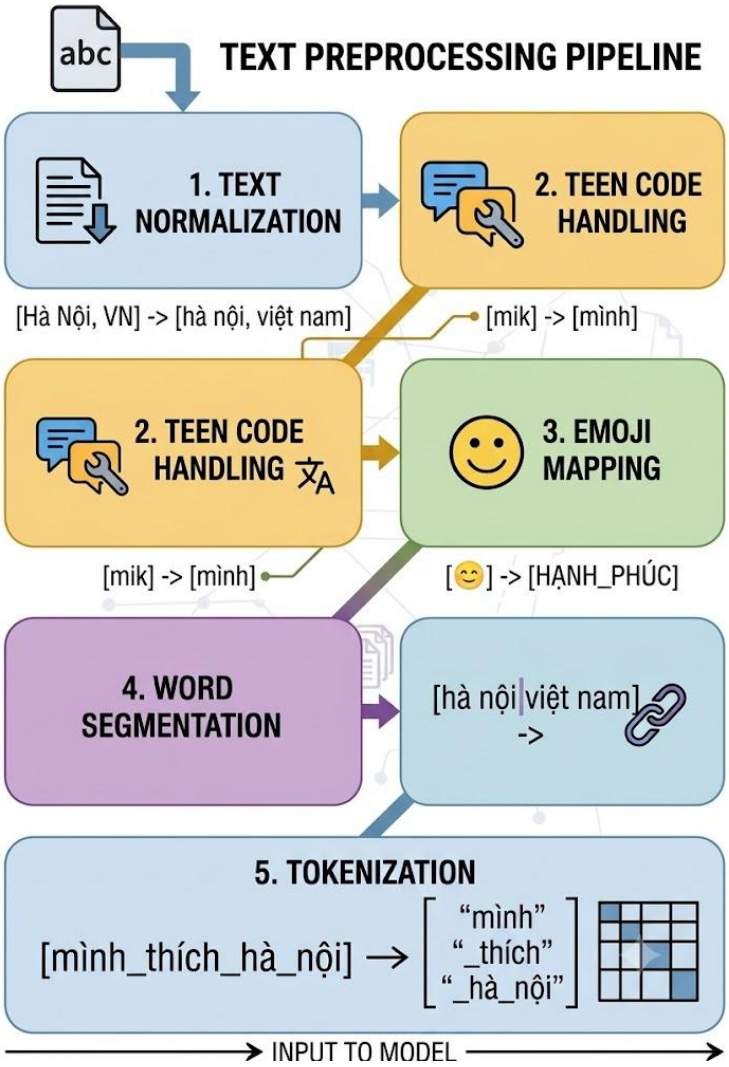

Figure 3.2: Five-Step Text Preprocessing Pipeline for Vietnamese Fintech Reviews

3.3.2 Text Normalization

The first step is text normalization: lowercasing all characters and applying Unicode NFC normalization. Lowercasing reduces vocabulary size by collapsing case variants into one token, which matters especially for English terms that appear in mixed capitalization. Unicode NFC normalization puts Vietnamese diacritics into a consistent composed form, so the same visible character is not treated as two different tokens because of different underlying encodings. Let r denote a raw review string.

The normalization function f_{norm} produces:

$$r' = f_{norm} = NFC(lowercase(r))$$

This step is applied before all subsequent preprocessing operations and is identical across all model architectures.

3.3.3 Vietnamese Teen Code Handling

The second step handles Vietnamese teen code, the informal phonetic shorthand common in Vietnamese social media and app store reviews. Teen code swaps standard Vietnamese characters for phonetically similar letters, numbers, or combinations that fall outside normal orthography. For example, "được" becomes "dc," "không" becomes "k" or "ko," and vowels are sometimes replaced with numbers. Native speakers read these easily, but standard NLP tools cannot parse them.

A mapping dictionary of common teen code substitutions was compiled and applied as a lookup-and-replace pass over the normalized text.. Let $\mathcal{D} = (t_i, s_i)_{i=1}^{M}$ denote the dictionary of M teen code-to-standard form mappings, where t_i is the teen code token and s_i is its standard Vietnamese equivalent. The teen code handling function applies:

$$r'' = f_{\text{teen}}(r') = \text{replace}(r', \mathcal{D})$$

This step reduces out-of-vocabulary rates for all models and improves the alignment between review text and the pre-training vocabulary of Transformer models.

3.3.4 Emoji Mapping

The third step converts emoji into explicit sentiment tokens. Emoji appear frequently in Vietnamese app reviews and carry clear sentiment signals, so removing them entirely would lose useful information. Instead of discarding them, each emoji is mapped to a descriptive sentiment token.

Let $\mathcal{E} = (e_j, \tau_j)_{j=1}^{K}$ denote the mapping of K emoji characters to sentiment tokens. The emoji mapping function applies:

$$r''' = f_{\text{emoji}}(r'') = \text{replace}(r'', \mathcal{E})$$

Mapping emoji to explicit tokens rather than removing them preserves the sentiment signal they carry while converting them into a form that standard tokenizers can process.

3.3.5 Word Segmentation

The fourth step is Vietnamese word segmentation using VnCoreNLP (Vu et al., 2018). Vietnamese separates syllables with spaces rather than words, so a word like "ngân hàng" (bank) appears as two separate tokens that are each ambiguous on their own. The segmenter identifies which adjacent syllables belong together and joins them with underscores, producing tokens like "ngân_hàng" that downstream models can treat as single units.

Let f_{seg} denote the VnCoreNLP word segmentation function. The segmented review is:

$$r'''' = f_{\text{seg}}(r''')$$

Word segmentation is applied before tokenization for recurrent models, which use a custom Vietnamese vocabulary built on segmented text. For Transformer models, the pre-trained PhoBERT tokenizer already incorporates word segmentation conventions from its pre-training corpus, so VnCoreNLP segmentation is applied consistently across all models to maintain a uniform preprocessing pipeline.

3.3.6 Tokenization

The fifth and final preprocessing step converts segmented text into token sequences suitable for model input. For recurrent models (LSTM, BiLSTM, GRU, LSTM+Attention), WordPiece tokenization is applied over the segmented vocabulary, producing integer token sequences that are padded or truncated to a fixed maximum length. For Transformer models (PhoBERT, XLM-RoBERTa), the respective pre-trained tokenizers are used: the PhoBERT tokenizer for PhoBERT and the XLM-RoBERTa sentencepiece tokenizer for XLM-RoBERTa. Both Transformer tokenizers produce input_ids, attention_mask, and token_type_ids tensors as required by the Hugging Face model interface.

3.4 Imbalance Handling Strategies

3.4.1 Motivation

As established in Chapter 2, the dataset contains a severe class imbalance with Negative reviews accounting for 81.3% of samples, Positive for 13.4%, and Neutral for only 5.4%. The resulting Negative-to-Positive ratio of approximately 6.08:1 and Negative-to-Neutral ratio of approximately 15.15:1 are sufficient to cause standard classifiers to collapse to majority-class prediction. Four strategies are evaluated to address this imbalance, each applied exclusively to the training set to avoid contaminating the evaluation.

3.4.2 Class-Weighted Loss

The first strategy assigns per-class weights to the cross-entropy loss function inversely proportional to class frequency, so that each class contributes equally to the total loss regardless of its prevalence in the training set. Let n_c denote the number of training samples in class c and N the total number of training samples across C classes. The weight for class c is:

$$w_c = \frac{N}{C \cdot n_c}$$

Applying these weights to the dataset yields $w_{\text{Negative}} = 0.41$, $w_{\text{Positive}} = 2.49$, and $w_{\text{Neutral}} = 6.21$. The weighted cross-entropy loss for a batch of B samples is:

$$\mathcal{L}\text{weighted} = -\frac{1}{B}\sum i = 1^{B}\, w_{y_i} \log p\,(y_i \mid x_i)$$

where y_i is the true label and $p(y_i \mid x_i)$ is the model's predicted probability for that label.

3.4.3 SMOTE Oversampling

The second strategy applies Synthetic Minority Over-sampling Technique (SMOTE) to the training feature space (Chawla et al., 2002). SMOTE generates synthetic minority class samples by interpolating between existing minority class instances in the embedding space rather than simply duplicating them. For each minority class sample x_i, a synthetic sample x_{syn} is generated as:

$$x_{\text{syn}} = x_i + \lambda \cdot (x_{knn} - x_i), \quad \lambda \sim \mathcal{U}(0,1)$$

where x_{knn} is a randomly selected sample from the k-nearest neighbors of x_i within the same class, and λ is a uniform random interpolation coefficient. In this study, SMOTE is applied to oversample both Positive and Neutral classes to 50% of the majority Negative class size.

3.4.4 Random Undersampling

The third strategy reduces class imbalance by randomly removing majority class samples from the training set until the Negative-to-Positive ratio is reduced to approximately 3:1. Let $n_{\text{Neg}}^{\text{orig}}$ be the original number of negative training samples. The target number of negative samples after undersampling is:

$$n_{\text{Neg}}^{\text{target}} = 3 \cdot n_{\text{Pos}}$$

Samples to be removed are selected uniformly at random from the Negative class without replacement. This approach discards a substantial portion of the available majority class training data but avoids the overfitting risk associated with extreme oversampling.

3.4.5 Hybrid Strategy

The fourth strategy combines random undersampling of the majority class with SMOTE oversampling of the minority classes. The majority class is undersampled to an intermediate size first, then SMOTE brings both minority classes closer to that level. The goal is to reduce the imbalance without relying too heavily on either technique alone: undersampling discards less data than it would on its own, and SMOTE does not need to generate as many synthetic samples, which limits overfitting risk.

3.5 Data Splitting

3.5.1 Chronological Split

The data is split chronologically rather than randomly. The earliest 80% of reviews form the training set (3,460 reviews) and the most recent 20% form the test set (865 reviews). This mirrors how a deployed model would actually be used: it trains on past reviews and is tested on reviews written after the training data was collected, so no future information leaks into training.

Let T_i denote the timestamp of review i, and let $T_{0.8}$ be the 80th percentile timestamp in the full dataset. The training and test sets are defined as:

$$\mathcal{D}\text{train} = (x_i, y_i) \mid T_i \leq T0.8, \quad \mathcal{D}\text{test} = (x_i, y_i) \mid T_i > T0.8$$

Imbalance handling is applied only to $\mathcal{D}$train, and all evaluation metrics are computed on $\mathcal{D}$test in its original unmodified form. This ensures that reported performance reflects the true class distribution the model would encounter in deployment.

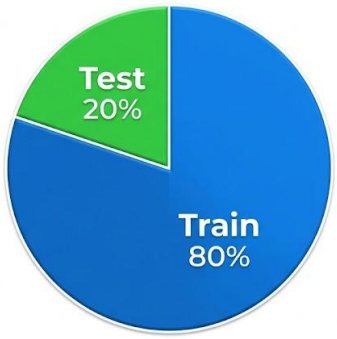

Figure 3.3: Chronological 80/20 Train-Test Split of the Momo Review Dataset

3.6 Model Architectures

3.6.1 Recurrent Models

Three baseline recurrent architectures are implemented and evaluated: LSTM, BiLSTM, and GRU. All three share the same embedding configuration ,a 300-dimensional trainable word embedding layer initialized randomly ,and differ only in their recurrent cell type and directionality.

The LSTM model stacks two LSTM layers with 128 hidden units each, with dropout of 0.3 applied between layers. At each time step t, the LSTM updates its hidden state h_t and cell state c_t according to:

$$f_t = \sigma\big(W_f[h_{t-1}, x_t] + b_f\big)$$
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$
$$\tilde{c}t = \tanh(W_c[ht - 1, x_t] + b_c)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}t$$
$$o_t = \sigma(W_o[ht - 1, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(c_t)$$

where f_t, i_t, $o_t$are the forget, input, and output gates respectively, σ denotes the sigmoid function, and $\odot$ denotes element-wise multiplication.

The BiLSTM model processes the input sequence in both forward and backward directions using separate LSTM layers with 128 units per direction, concatenating the forward hidden state $\overrightarrow{h}_t$ and backward hidden state $\overleftarrow{h}_t$ at each position:

$$h_t = \left[\overrightarrow{h}_t ; \overleftarrow{h}_t\right] \in \mathbb{R}^{256}$$

The GRU model replaces the LSTM cell with a Gated Recurrent Unit that uses two gates ,a reset gate r_t and an update gate z_t ,to control information flow:

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$
$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$
$$\tilde{h}t = \tanh(W_h[r_t \odot ht - 1, x_t] + b_h)$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

For all three recurrent models, the final hidden state is passed through a linear classification head with softmax activation to produce a probability distribution over the three sentiment classes.

3.6.2 LSTM with Attention

The fourth recurrent model augments the BiLSTM architecture with a self-attention mechanism that computes a weighted sum of all hidden states rather than relying solely on the final hidden state. Let $H = [h_1, h_2, \ldots, h_T] \in \mathbb{R}^{T \times 256}$ denote the matrix of BiLSTM hidden states. The attention weights α_t and context vector c are computed as:

$$e_t = W_a h_t + b_a$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^{T} \exp(e_j)}$$

$$c = \sum_{t=1}^{T} \alpha_t \, h_t$$

The context vector c is then passed to the classification head. The attention mechanism allows the model to selectively weight the contribution of different positions in the review when forming the final representation, which is particularly useful for aspect-related sentiment where the relevant information may be concentrated in a small portion of the text.

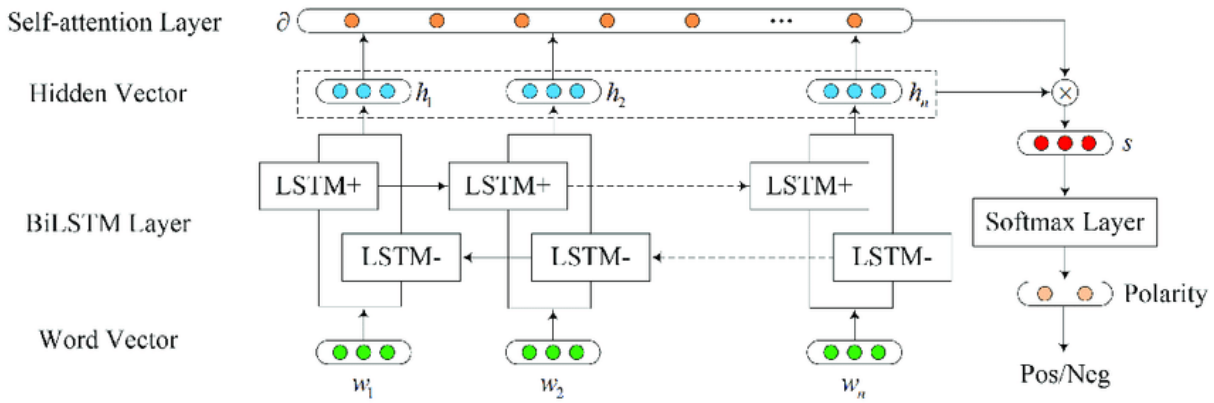

Figure 3.4: BiLSTM with Self-Attention Architecture for Sentiment Classification

Source: Self-Attention-Based BiLSTM Model for Short Text Fine-grained Sentiment Classification, ResearchGate

3.6.3 PhoBERT

PhoBERT is a RoBERTa-based Transformer model pre-trained on 20GB of Vietnamese text (Nguyen & Nguyen, 2020). The model consists of 12 Transformer encoder layers with 768 hidden dimensions, 12 attention heads, and approximately 135 million parameters. For sentiment classification, the [CLS] token representation from the final encoder layer is extracted and passed through a linear classification head:

$$h_{\text{CLS}} = \text{PhoBERT}(x)[0] \in \mathbb{R}^{768}$$

$$\hat{y} = \text{softmax}(W_c h_{\text{CLS}} + b_c)$$

Fine-tuning is performed with a learning rate of 2×10^{-5}, batch size of 8, and early stopping with patience 5. The entire encoder is fine-tuned jointly with the classification head rather than freezing the lower layers, as this approach consistently produces better results for domain-specific text classification tasks.

3.6.4 XLM-RoBERTa

XLM-RoBERTa is a multilingual RoBERTa model pre-trained on 2.5TB of text spanning 100 languages using a shared sentencepiece vocabulary of 250,000 tokens (Conneau et al., 2020). The architecture is identical to PhoBERT in terms of layer count and hidden dimensions ,12 layers, 768 hidden dimensions, 12 attention heads ,but the substantially larger and more diverse pre-training corpus gives it broader cross-lingual transfer capabilities at the potential cost of language-specific representation quality. The fine-tuning procedure mirrors that used for PhoBERT, enabling direct architectural comparison between the two models.

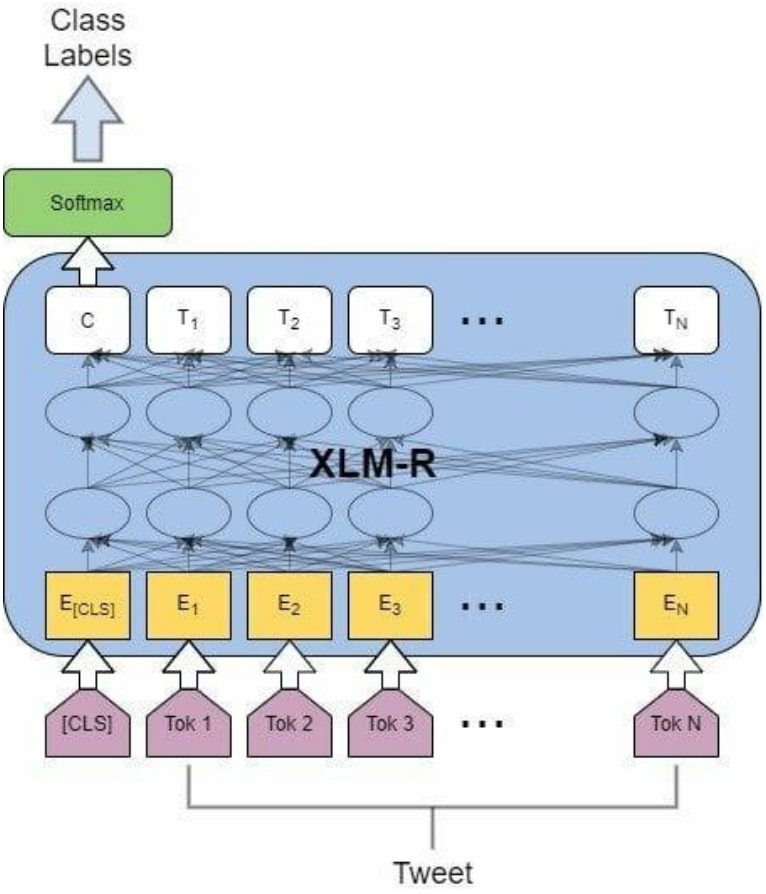

Figure 3.5: XLM-RoBERTa architecture

Source: TechSSN at SemEval-2023 Task 12: Monolingual Sentiment Classification in Hausa Tweets, ResearchGate

3.7 Multi-Task Learning Configuration

3.7.1 Shared Encoder with Task-Specific Heads

The multi-task learning setup extends the PhoBERT and XLM-RoBERTa single-task models by adding two auxiliary classification heads alongside the primary sentiment head. The shared Transformer encoder produces a single [CLS] representation h_{CLS} that is simultaneously used by all three heads:

$$\hat{y}\text{sent} = \text{softmax}(W_s h\text{CLS} + b_s) \in \mathbb{R}^3$$
$$\hat{y}\text{rating} = \text{softmax}(W_r h\text{CLS} + b_r) \in \mathbb{R}^5$$
$$\hat{y}\text{aspect} = \text{softmax}(W_a h\text{CLS} + b_a) \in \mathbb{R}^8$$

where the sentiment head classifies into 3 sentiment classes, the rating head classifies into 5 star rating classes, and the aspect head classifies into 8 fintech aspect categories.

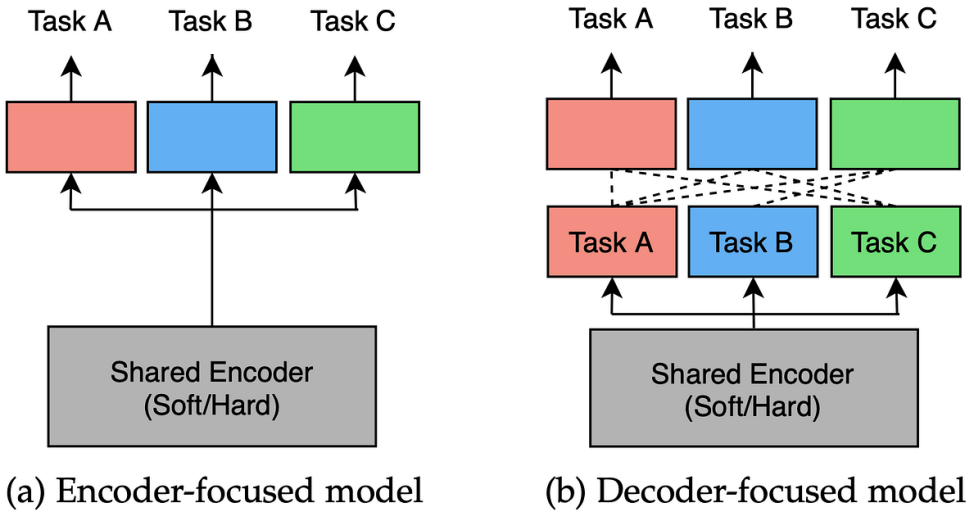

Figure 3.6: Multi-Task Learning Architecture with Shared Encoder and Three Classification Heads

Source: Introduction to Multi-Task Learning, Medium

3.7.2 Combined Loss Function

The training objective for the multi-task models is a weighted sum of the three task-specific cross-entropy losses:

$$\mathcal{L}\text{total} = \alpha \cdot \mathcal{L}\text{sentiment} + \beta \cdot \mathcal{L}\text{rating} + \gamma \cdot \mathcal{L}\text{aspect}$$

where:

$$\mathcal{L}\text{task} = -\frac{1}{B}\sum i = 1^B \log p\,(\hat{y}\text{task}, i = y\text{task}, i \mid x_i)$$

The loss weights are set to $\alpha = 1.0$, $\beta = 0.5$, and $\gamma = 0.3$, reflecting the primacy of the sentiment classification objective and the auxiliary role of rating prediction and aspect categorization. The higher weight on sentiment ensures that the shared encoder prioritizes representations useful for the primary task, while the auxiliary weights are set low enough to regularize without dominating the gradient signal.

3.8 Training Configuration

3.8.1 Optimization and Regularization

All models are trained using the Adam optimizer with gradient clipping at a maximum norm of 1.0 to prevent gradient explosion, which is particularly important for recurrent models on imbalanced datasets. Training runs for a maximum of 50 epochs with early stopping based on validation Macro-F1, with a patience of 5 epochs ,training stops if validation Macro-F1 does not improve for 5 consecutive epochs. The model weights from the epoch with the highest validation Macro-F1 are retained for evaluation.

Batch sizes differ by model family: recurrent models use a batch size of 16, while Transformer models use a batch size of 8 due to the larger memory footprint of the 12-layer encoders. Dropout of 0.3 is applied between recurrent layers in all RNN-based models. For Transformer models, the pre-trained dropout rates within the encoder layers are preserved during fine-tuning.

Table 3.2: Training Hyperparameters by Model Family

Hyperparameter	RNN Models	Transformer Models
Optimizer	Adam	Adam
Learning rate	1×10^{-3}	2×10^{-5}
Batch size	16	8
Max epochs	50	50
Early stopping patience	5	5
Gradient clipping	1.0	1.0
Dropout	0.3	Pre-trained defaults
Embedding dimension	300	768

3.9 Evaluation Metrics

3.9.1 Macro-F1

Macro-F1 is the primary evaluation metric throughout this study because it weights all classes equally regardless of their frequency, penalizing models that achieve high overall accuracy by ignoring minority classes. For a classification problem with C classes, Macro-F1 is defined as the unweighted average of the per-class F1 scores:

$$\text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C} F\,1_c$$

where the F1 score for class ci s:

$$F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$$

In the context of this study with three sentiment classes ($C = 3$), a model that predicts all samples as Negative achieves Macro-F1 of approximately 0.28 despite achieving over 80% accuracy, because its F1 for the Positive and Neutral classes is zero. Macro-F1 therefore provides a more honest measure of model utility than accuracy alone.

3.9.2 Weighted-F1 and Balanced Accuracy

Weighted-F1 computes the F1 score for each class and takes a weighted average proportional to class frequency:

$$\text{Weighted-F1} = \sum_{c=1}^{C} \frac{n_c}{N} \cdot F1_c$$

While Weighted-F1 is less sensitive to minority class performance than Macro-F1, it provides a useful complement by reflecting the performance distribution that would be observed on the natural class distribution. Balanced Accuracy computes the average of per-class recall rates, providing another class-balanced alternative to overall accuracy:

$$\text{Balanced Accuracy} = \frac{1}{C} \sum_{c=1}^{C} \text{Recall}_c$$

3.9.3 Auxiliary Task Metrics

For the multi-task learning evaluation, two additional metrics are reported for the auxiliary tasks. Aspect categorization performance is measured using Macro-F1 over the eight aspect classes, computed identically to the sentiment Macro-F1. Rating prediction performance is measured using Mean Absolute Error (MAE), which captures the average deviation between predicted and true star ratings on the 1–5 scale:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^{N} |\hat{r}_i - r_i|$$

where $\hat{r}_i$ is the predicted rating class and r_i is the true rating. Lower MAE indicates better rating prediction accuracy, with a value of 0 corresponding to perfect prediction and a value of 4 corresponding to the maximum possible error.